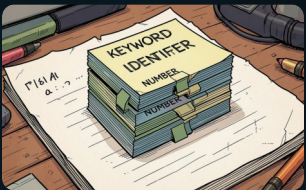


Fundamentos del Análisis Léxico: de caracteres a tokens

El análisis léxico es la primera fase de un compilador: transforma una secuencia de caracteres en una secuencia de tokens significativos. Su objetivo es reconocer lexemas (la subcadena concreta) y asignarles un token (su categoría o tipo). Esta fase filtra espacios, comentarios y comprime la entrada para las fases siguientes (análisis sintáctico y semántico).

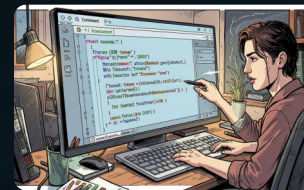
Conceptos clave:

- Lexema: la cadena concreta extraída (ej. "while", "123", "variable_name").
- Token (tipo): la categoría asignada al lexema (ej. KEYWORD, NUMBER, IDENTIFIER).
- Tokenización: proceso de escaneo y reconocimiento incremental de lexemas en la entrada.



Salida del lexer

Flujo compacto de tokens con metadatos (tipo, lexema, posición) que alimenta al parser.



Preprocesamiento

El lexer elimina comentarios y normaliza espacios; puede también manejar literales y escapes.

❏ Importante: los tokens suelen describirse mediante lenguajes regulares —por eso los autómatas finitos son el modelo matemático adecuado.

Autómatas Finitos: modelo formal para el análisis léxico

Un Autómata Finito Determinista (AFD) se define formalmente como la tupla $(Q, \Sigma, \delta, q_0, F)$: Q = conjunto finito de estados; Σ = alfabeto; $\delta: Q \times \Sigma \rightarrow Q$ = función de transición; q_0 = estado inicial; F = estados de aceptación. Un AFD procesa la cadena símbolo a símbolo y acepta si termina en un estado de F .



Relación con lenguajes regulares

Los lenguajes que describen tokens son regulares; por tanto, existe un AFD que los reconoce. Esto hace a los AFDs suficientes y eficientes para el lexer.



AFD vs AFND

Un AFND permite múltiples transiciones y ϵ -transiciones; es más expresivo como construcción, pero cualquier AFND puede transformarse en un AFD equivalente (algoritmo de subconjuntos).



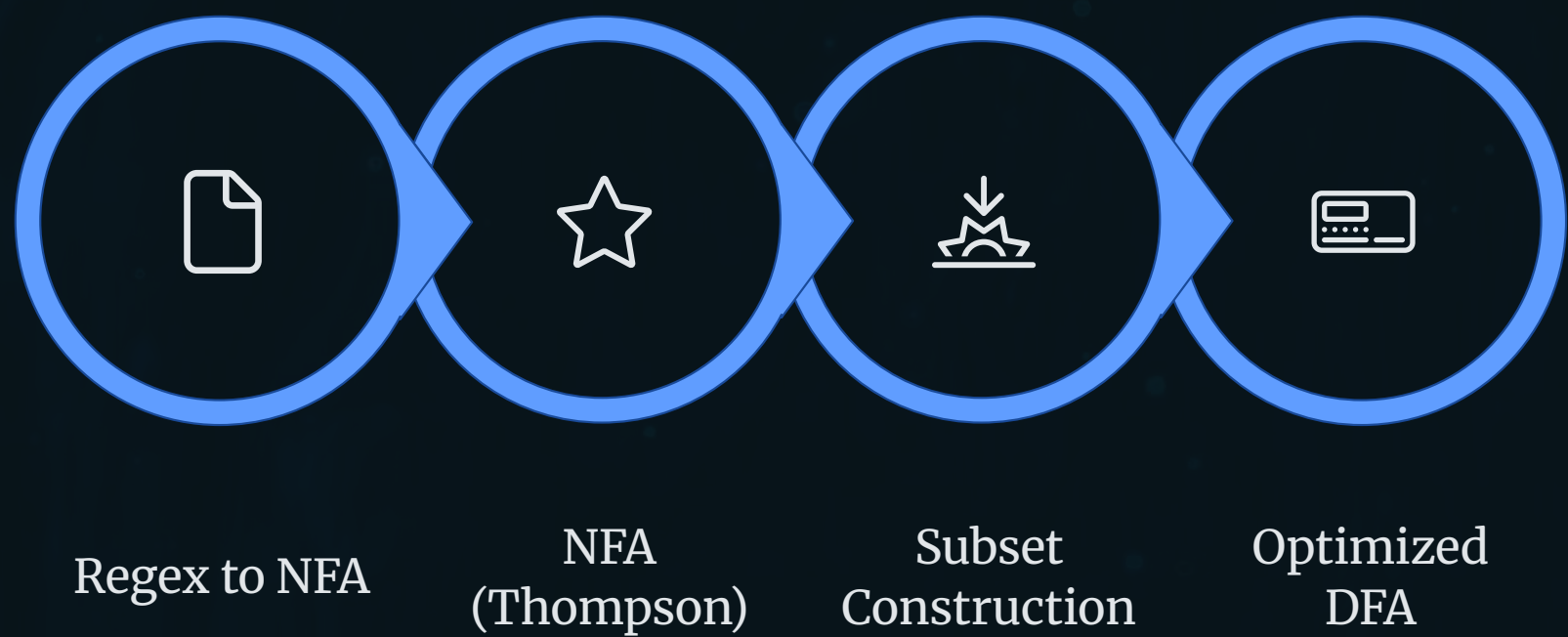
Implementación práctica

Los lexers implementan AFDs compactos (tablas δ o código con switch). Ventaja: ejecución en tiempo lineal respecto al número de símbolos.

Ejemplo conceptual: un AFD que reconoce números enteros (alfabeto $\Sigma = \{0..9\}$) avanzará por estados hasta consumir todos los dígitos y alcanzar un estado de aceptación que emite el token NUMBER.

Expresiones regulares y la cadena lógica: ER → AFND → AFD → Lexer

Las expresiones regulares (ER) son una notación declarativa para describir patrones léxicos: concatenación, alternancia (|), y cerradura (*) permiten definir conjuntos de cadenas. Su poder expresivo coincide con los lenguajes regulares.



Proceso práctico y consideraciones:

- Construcción: ER → AFND (algoritmo de Thompson) genera un autómata no determinista sencillo de construir.
- Determinización: AFND → AFD (algoritmo de subconjuntos) produce un autómata determinista que el lexer puede ejecutar eficientemente.
- Minimización / optimización: reducción de estados para tablas de transición compactas y menos memoria.
- Manejo de ambigüedades: reglas de prioridad (ej. longest-match, prioridad de tokens) y acción semántica al aceptar un token.

Paso 1 — Definir patrones

Especificar ER para cada clase léxica: identificadores, números, literales, operadores, etc.

Paso 2 — Generar autómatas

ER → AFND (Thompson) → AFD (subconjuntos) → minimizar si procede.

Paso 3 — Integrar en el lexer

Ejecutar el AFD sobre la entrada, aplicar longest-match y emitir tokens con posición y tipo.

📌 Consejo para la práctica: usar generadores de lexers (Flex, re2c, herramientas del lenguaje) acelera el desarrollo, pero comprender la transformación ER→AFD es esencial para depurar patrones y errores de tokenización.